Möbius

A ring of containers, the class it carries, and the price of a centre

Claude, 7 September 2026

Three documents in this repository argue that a federation is a diagram of morphisms between containers, that its graph has cycles, and that the obstruction to assembling it is a class in H^1 carried by the fundamental group. This note is about the program that tests those claims, and it is written in the language of containers rather than the language of sheaves.

The program is a ring of servers. Each server is a container, and each server is also a morphism between containers. Nothing in it is a simulation of a ring. The ring is made of processes, and the arithmetic that produces the invariant happens in the backward maps of real morphisms crossing real boundaries.

Two passes built it. The first pass built the base ring, found the missing trace as a compiler diagnostic, and established that the four repairs of the sheaf note are four fibres of one container. The second pass built the double cover and the deck transformation, which is where the price of a centre becomes a number that can be printed. This note covers both, and marks which is which.

1 The container, and the node that is also a morphism

A container is a pair. There is a type of shapes, and for each shape a type of positions over that shape. The repository reads the pair as an interface. A shape is a prompt the container accepts. The positions over a shape are the valid responses to that particular prompt. Writing the container as $P \triangleleft R$ makes P the prompts and R the dependent family of responses.

A morphism of containers is two maps, and they run in opposite directions. The forward map sends a prompt on the source interface to a prompt on the target interface. The backward map sends a response to the target prompt back to a response to the source prompt. Prompts move forwards. Responses move backwards.

The design decision the whole program rests on. Each node *has* a container, which is its interface. Each node also *is* a morphism, because it answers a delegating prompt by issuing a prompt to its neighbour and re-expressing the reply in its own frame. The transition function of the cover is applied in the backward map, and in no other position.

That last sentence is a constraint and not a preference. A prompt in this program carries no value. It says *tell me your local value*, and nothing more. Had a value travelled outwards on the prompt, the transition function would have been applied in the forward map, and the forward map is the wrong one. Descent data acts on sections. Sections come back. So the sign lives in the backward map, where the type of the function is a map from responses to responses, and where there is no way to apply it to a prompt. Appendix A shows the compiler being asked to confirm that.

2 The cover, and the class it carries

Cover the circle with three arcs, each slightly over a third. Every pairwise overlap is then a single arc, and the triple overlap is empty. Take coefficients in $\mathbb{Z}/2 = \{+1, -1\}$, written multiplicatively. The transition data is one sign per overlap, which is one sign per edge of the ring.

The rank computation is small enough to state in full. Both cochain groups are $(\mathbb{Z}/2)^3$, the second is zero, the coboundary of a 0-chain is $(d_0 a)_{ij} = a_i a_j$, and the kernel of d_0 is the constants. So the group of coboundaries has rank two, every 1-cochain is a cocycle because d_1 lands in zero, and the quotient is $\mathbb{Z}/2$. The program prints it.

```
THE NERVE

  arcs                  3
  dim C0, C1, C2        3, 3, 0
  Z1 = C1               rank 3   (d1 lands in 0)
  B1 = im d0            rank 2   (ker d0 = the constants)
  H1 = Z1/B1            Z/2

  The triple overlap is empty, so the cocycle condition is VACUOUS.
  This demonstrates monodromy. It does not exercise coherence.
```

The subgroup of coboundaries is exactly the set of edge assignments whose product is $+1$. There are therefore two classes and no more. Product $+1$ is the cylinder, and product -1 is the Möbius band. The arc count is load bearing. With two arcs the overlap has two components and the rank computation is different, and with four arcs the triple overlap is non-empty and the cocycle condition stops being vacuous.

Flipping one sign moves the ring from one class to the other with no change to any code. That is what it means for H^1 to classify the descent data rather than describe the space.

```
THE CLASS   [coordinator — this program did the arithmetic]

  edges         +,+,-
  product       -1
  class         mobius band
  global sections of the twisted sheaf: 0

  The qualifier is not decoration. The constant sheaf Z/2 always has
  its two constant sections. What is obstructed is the sheaf glued by
  these transitions.
```

The qualifier is worth pausing on. The constant sheaf keeps its two constant sections in both classes. What has no global section in the non-trivial class is the sheaf glued by these transition functions. A program that printed *zero sections* without naming which sheaf would be making a false claim, and the tag in square brackets is the subject of Section 7.

3 The missing trace, as a compiler diagnostic

Sequential composition of containers, written $\triangleleft$, builds trees. A federation is a ring, and a ring is not a tree. The first pass put that claim to the compiler rather than to the reader. Three leaves are declared, one per server. A path of three composes without complaint, because a path is a tree.

```
const n0 = leaf<NodeC>("n0", 8811, decoders);
const n1 = leaf<NodeC>("n1", 8812, decoders);
const n2 = leaf<NodeC>("n2", 8813, decoders);
```

```
const path = seqC(n0, seqC(n1, n2));   // fine: a path is a tree
const ring = seqC(n2, ring);           // TS7022: referenced in its own initializer
```

Removing the `@ts-expect-error` directive that guards the second declaration produces this, which is the diagnostic itself and not a paraphrase of it.

```
TS7022 [ERROR]: 'ring' implicitly has type 'any' because it does not have a type
annotation and is referenced directly or indirectly in its own initializer.
const ring = seqC(n2, ring);
      ~~~~
    at file:///Users/robin/git/gold/federation/mobius-type-tests.ts:50:7

TS2448 [ERROR]: Block-scoped variable 'ring' used before its declaration.
const ring = seqC(n2, ring);
                  ~~~~
    at file:///Users/robin/git/gold/federation/mobius-type-tests.ts:50:23
```

The obstruction can then be bought back with a thunk. The annotation supplies what inference could not find, so the following compiles. It also does not terminate.

```
const lazyRing: Runner<NodeC> = {
  label: "ring",
  run: <S extends { tag: string }>(s: S, d: number) => lazyRing.run(s as never, d),
};
```

The finding of the first pass. Containers type the interface and say nothing about the well-foundedness of the composite. The interface here is well typed. The composite is not well founded. Both facts are checked by the same compiler run, and the second is invisible to it.

4 The four repairs are four fibres of one container

The sheaf note lists four ways to repair a cyclic federation. Orient it by electing a centre. Unroll it to a finite approximation. Ask for a bounded transport. Detect the cycle and stop. The program implements all four, and none of them is a change of program. They are four fibres of the same container, differing only in the prompt and in one case only in the response type.

fibre	base case	what it is
`Ask`	fuel 0, answer locally	bounded transport, and the value comes back negated
`Unroll`	fuel 0, `Exhausted`	F^n of bottom, which is bottom, for every n
`Orient`	none	the unbounded one, which does not return
`Walk`	sees itself in `visited`	cycle detection, and what it reports is the class

Three of the four differ from each other only in the prompt. The prompt for `Unroll` carries the same fuel as the prompt for `Ask`, and the difference between them is entirely in the response type, which admits an `Exhausted` constructor.

Fuel rides in the prompt rather than in the depth counter that crosses the wire in an HTTP header. That is not tidiness. A header cannot be a type, and putting the fuel in the prompt is what allows the type test to assert at compile time that nothing is carried at fuel zero.

The `Walk` fibre deserves its own sentence, because of where it does its arithmetic. The node that recognises itself in the visited list returns the empty product. Every frame on the way back

multiplies by its own edge transition. So detecting the cycle and computing the first cohomology class are the same act, and the holonomy accumulates entirely in the backward maps.

```
  WALK   [in the ring — the backward maps did the arithmetic]

    loop        [0,1,2,0]
    holonomy    -1

    Detecting the cycle and computing H1 are the same act. The node
    that sees itself returns the empty product, and every frame on the
    way back multiplies by its own edge.
```

5 Infinite regress is not deadlock

The unbounded fibre has no base case, no fuel and no memory of where it has been. The prose about it must be exact, because two different failures are easy to confuse.

Nothing in the ring deadlocks. Nothing is ever blocked waiting on a peer. Each hop is a fresh process that issues one prompt and waits for one reply, and every reply that is owed will eventually be produced, given unbounded resources. What the ring does is regress without bound. The following was measured while one unbounded request was outstanding, by counting live delegating processes once every two seconds.

```
  t+1: 3 delegate processes live
  t+2: 123 delegate processes live
  t+3: 238 delegate processes live
  t+4: 334 delegate processes live
--- after the request was abandoned ---
  still live: 558
  Orient trace lines in the log: 558
```

Read the last two lines together. The client had already given up, and three seconds later the count was still climbing. It reached 558, and the log recorded exactly 558 outbound hops. The two numbers agree because not one of those hops has returned. Abandoning the request does not stop the ring. Only stopping the servers does, and it stops silently, because a termination signal does not give the handlers a chance to report.

Observation. *The response type of the unbounded fibre is inhabited and total. No total program inhabits it. The type checker, the serving loop and the delegating call all accept it without complaint. This is the same finding as Section 3, arrived at from the runtime rather than from the compiler.*

6 The fifth fibre, and the price of a centre

The sheaf note ends by listing a repair it did not carry out. Pass to the universal cover. On a simply connected base the locally constant classes vanish, because there is no fundamental group left to represent them. The centre becomes available, and the price is infinitely many copies of the federation.

With coefficients in $\mathbb{Z}/2$ that price is finite. The relevant cover has order two, so infinitely many copies is two copies and six nodes, and the whole universal cover fits on one machine. The second pass built it.

6.1 The pullback, and the trap in it

The n-fold cover of the circle pulls a 3-arc cover back to $3n$ arcs. The upstairs transition data is the downstairs data repeated, so $\varepsilon_i = g_{i \bmod 3}$. Six servers are started, carrying the base data

twice.

```
up: 6 arcs on 8811 8812 8813 8814 8815 8816   signs +,+,−,+,+,−   (the 2-fold cover of +,+,−)
```

Now the trap, which is the reason the first pass deferred this rather than guessing. The holonomy upstairs is $(\prod g)^2$, and that is $+1$ in *both* classes. Six hops therefore trivialise the twist on the cylinder as well as on the Möbius band. A demonstration that walked six hops and reported that the value came back unchanged would print the same thing in both classes. It would prove nothing while appearing to prove everything.

The correct reading of that $+1$. It is not a result about the twist. It is the statement $p^*\alpha = 0$. The class corresponds to a homomorphism $\alpha : \pi_1(S^1) = \mathbb{Z} \to \mathbb{Z}/2$, which is $m \mapsto m \bmod 2$. The fundamental group of the double cover maps onto $2\mathbb{Z}$, and α is trivial there. So a section exists upstairs, always, and that is the whole reason there is anything to compare.

6.2 The deck transformation

Build the section from node 0 outwards, with $s_0 = +1$ and $s_{i+1} = \varepsilon_i \cdot s_i$. Both classes give a section that closes.

```
mobius    ε = +,+,−,+,+,−      s = +1 +1 +1 −1 −1 −1      σ*s = −s
cylinder  ε = +,+,+,+,+,+      s = +1 +1 +1 +1 +1 +1      σ*s = +s
```

What separates them is the deck transformation $\sigma(i) = i + 3 \bmod 6$. It shifts by the size of the base rather than by half the ring, because the deck group of the n-fold cover of the circle is $\mathbb{Z}/n$ and this is its generator. In the trivial class the two sheets agree. In the non-trivial class they do not, and the ratio between them is the product over the base.

The result. The deck transformation recovers the base class from a cover on which nothing is obstructed. A section exists upstairs and it is not deck-invariant, so the federation is consistent only by being two federations that disagree with each other. That is a precise statement of what a centre costs.

The command prints the pulled-back data, then the section, then the fact that it glues in both classes, and only then the deck transformation. That order is deliberate. A reader who sees *it glues* first and stops has learned the wrong thing.

```
degree          2
pulled back     +,+,−,+,+,−   eps_i = g_(i mod 3)

the section upstairs, from node 0 with s_0 = +1

  s         +1  +1  +1  −1  −1  −1   installed, one value per node

  closure   eps_5 . s_5 = +1 = s_0   it glues
  cuts      every cut gives the same section
  product   +1   upstairs, over all 6 edges

+1 IN BOTH CLASSES. Six hops trivialise the twist on the cylinder
too, so nothing above this line separates them, and a naive
demonstration that stopped here would print the same thing either
way. This is p*alpha = 0 — alpha is m |-> m mod 2 and the cover's
pi_1 is 2Z — and it is the reason a section exists at all, not a
result about the twist.

the deck transformation, sigma(i) = i + 3 mod 6
```

```
sigma*s   -1  -1  -1  +1  +1  +1
ratio     -1   the same at every node
```

Two lines in that output are checks and not decoration. The closure line confirms that the last edge transition carries the last value back to the first. The cuts line confirms that all six ways of cutting the ring and rooting the remainder give the same section, which is what *it glues* means when it is verified rather than asserted. Downstairs on the Möbius band no two cuts agree.

6.3 The fifth fibre, and its two backward maps

The comparison so far was done by the command-line program, which polled every arc and did the arithmetic itself. That is legitimate here only because upstairs there is nothing to obstruct it. Doing it in the ring is a separate fibre, and designing it correctly is the whole content of the second pass.

The obvious design fails, one level below where the trap was expected. Transport with fuel three from node i returns the value at node $i + 3$ multiplied by the three edge transitions between them. The section satisfies $s_{i+3} = (\prod g) \cdot s_i$ by construction, so what comes back is $(\prod g)^2 \cdot s_i$, which is s_i in both classes. Transport agrees with itself. A fibre reporting only that would print the same thing either way, which is the trap again in a smaller form.

The resolution is that node i has two routes into the frame of node $i + 3$, and they are different maps.

- Walk three hops in the ring and multiply by the edge transition at every frame. That is the backward map of `Ask`, unchanged.
- Use the covering map. Nodes i and $i + 3$ lie over the same arc of the base, so the covering projection identifies their fibres, and that identification is the identity rather than a transition function.

So the `Deck` fibre is the transport recursion carrying two values back along one walk under two backward maps. The value called `transported` is multiplied by the edge transition at every frame. The value called `theirs` is relayed untouched. The nodes in between relay it without reading it, which they must, because they lie over a different arc of the base and it is not a value in their frame.

```
Deck: async (p, depth) => {
    if (p.fuel <= 0) {
      return { node: me, antipode: me, transported: value, theirs: value, agree: true };
    }
    const raw = await hop({ tag: "Deck", dir: p.dir, fuel: p.fuel - 1 }, p.dir, depth);
    const r = decodeDeck(raw);
    // — f, twice: eps for the transport, the identity for the covering map —
    const transported = (eps(p.dir) * r.transported) as Sign;
    return {
      node: me,
      antipode: r.antipode,
      transported,
      theirs: r.theirs,
      agree: transported === r.theirs,
    };
  },
```

Both maps are honest and they disagree, and what they disagree by is the class of the base. Here is the ring computing it, in the non-trivial class and then in the trivial one.

```
  THE DECK TRANSFORMATION   [in the ring — the backward maps did the arithmetic]

    node 0 walks 3 hops cw and arrives at node 3, the other
    sheet over the same arc of the base

    transported   +1   eps at every frame, the same backward map as Ask
    theirs        −1   relayed untouched; the covering map identifies the frames
    ratio         −1   = the product over the BASE

    the two backward maps disagree, and what they disagree by is the
    class of the base. Neither map is wrong.
```

Then `./mobius flat`, which sets the base to the trivial class and changes no code. The same command on the same six servers now reports the other answer.

```
    transported   +1   eps at every frame, the same backward map as Ask
    theirs        +1   relayed untouched; the covering map identifies the frames
    ratio         +1   = the product over the BASE

    the two backward maps agree. The section is deck-invariant and
    descends to the base.
```

The trace of that walk is worth reading, because it shows the recursion and the fuel descending, and it shows the six local prompts the command-line program issued first in order to count how many arcs were listening.

```
↓ node0     Local
↑ node0     {node, value}
↓ node1     Local
↑ node1     {node, value}
↓ node2     Local
↑ node2     {node, value}
↓ node3     Local
↑ node3     {node, value}
↓ node4     Local
↑ node4     {node, value}
↓ node5     Local
↑ node5     {node, value}
↓ node0     Deck dir="cw" fuel=3
  ↓ node1     Deck dir="cw" fuel=2
    ↓ node2     Deck dir="cw" fuel=1
      ↓ node3     Deck dir="cw" fuel=0
      ↑ node3     {node, antipode, transported, theirs, …}
    ↑ node2     {node, antipode, transported, theirs, …}
  ↑ node1     {node, antipode, transported, theirs, …}
↑ node0     {node, antipode, transported, theirs, …}
```

One point of hygiene. The section is installed into the nodes, one value per node, rather than merely printed. With the nodes still holding a constant the comparison collapses back into the plain holonomy of the base. The number printed would be the same, and the demonstration would be a different and weaker one.

What this does not claim. The obstruction has not gone away. It has been pulled back to a cover where it is zero, and downstairs it is exactly where it was. Six hops do not trivialise the twist, because they do the same thing on the cylinder, and the program says so in its output next to the number. In `seven-federations.pdf` this outcome is the Ridda, where the root transferred and there ended up being two answers, each internally coherent.

7 What is demonstrated, and what is not

The program marks its own commands. A command that polled every node and did the arithmetic in the command-line program is tagged `[coordinator]`. A command whose answer was accumulated in the backward maps of the ring, with the client standing at one node knowing nothing, is tagged `[in the ring]`. That distinction is not cosmetic. Electing a coordinator *is* the orienting repair, so a command that quietly does it has answered a different question from the one asked.

Five further limits are stated in the repository, and they belong in this note too.

The cocycle condition is vacuous. The triple overlap is empty, so the condition $g_{ij}g_{jk} = g_{ik}$ has no instance. The program exhibits monodromy. It does not exercise coherence, which is the centrepiece of the sheaf note. A fourth arc would create a genuine triple overlap, and that is the largest remaining distance between what the documents claim and what the code shows.

The coefficients are abelian. Order never mattered in either class, so the path comparison in the program is a path-independence check and not a commutativity one. Separating those two would need non-abelian coefficients, which would give up $H^1(S^1; \mathbb{Z}/2) = \mathbb{Z}/2$ and with it the two-class arithmetic.

Every element is its own inverse. The relation $g_{ji} = g_{ij}^{-1}$ is therefore invisible, and the two directions round the ring give the same holonomy. This model cannot distinguish a loop from its inverse.

The forward map is near-identity. Every edge is a vertical morphism, an element of the automorphism sheaf. That is exactly right for descent, and it is a degenerate corner of the container formalism. Giving the forward map real work is a documented extension and not a fix.

The obstructed object is the twisted sheaf. The constant sheaf keeps its two constant sections in both classes, and Section 2 says so in the output rather than in a footnote.

8 What remains

A guarded fibre returning a `Later` constructor, differing from the unrolling fibre in the response type rather than in the prompt. A fourth arc, where the cocycle condition would start to bite. A non-trivial forward map, so that both directions of the morphism are visible at once. The fourth arc is the one worth doing next, because it is the only one of the three that closes a gap between a claim and its demonstration.

A Containers in TypeScript, and the dependent lookup

The obstacle is easy to state. Sequential composition of containers has a shape type which is a dependent pair,

$$(s : c.\mathrm{Shape}) \times (c.\mathrm{Pos}\ s \to d.\mathrm{Shape}),$$

and TypeScript has no dependent sum. An earlier note in this line of work concluded that this was the hard part and left it there.

The conclusion is true of a dependent sum over an arbitrary index. It is false over a *finite* index, and every container in this program has a finite shape type, because a shape type here is a tagged union. Over such an index,

$$\sum_{s:S} B\,s \;=\; \bigsqcup_{s \in S} \{s\} \times B\,s,$$

and a conditional type in TypeScript distributes over a union. So the whole construction works, provided a container is presented not as a pair of a shape type and a position family, but as its *graph*, meaning the union of its fibres.

```
/** One fibre: a shape, and the type of positions over it. Never constructed —
 *  this is a type-level record, the object-language of Fam(Set^op). */
export interface Fib<S, P> {
  readonly shape: S;
  readonly pos: P;
}

export type Cont = Fib<unknown, unknown>;

export type Shape<C extends Cont> = C extends Fib<infer S, unknown> ? S : never;
```

Nothing of type `Fib` is ever built at runtime. A container is then a union of such records, which is exactly how the ring's node declares itself.

```
export type NodeC =
  | Fib<{ tag: "Local" }, LocalReport>
  | Fib<{ tag: "Edges" }, EdgeReport>
  | Fib<{ tag: "Configure"; edges: RingSigns; value: Sign }, Ack>
  | Fib<{ tag: "Ask"; dir: Dir; fuel: number }, Carried>
  | Fib<{ tag: "Unroll"; dir: Dir; fuel: number }, Bounded>
  | Fib<{ tag: "Orient"; dir: Dir }, Carried>
  | Fib<{ tag: "Walk"; dir: Dir; visited: RingId[] }, LoopClosed>
  | Fib<{ tag: "Deck"; dir: Dir; fuel: number }, DeckReport>;
```

A.1 The dependent lookup

The position family applied to one shape is a conditional type, and the direction of the test in it is the one thing that is easy to get wrong.

```
export type PosAt<C extends Cont, S> =
  C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) : never;
```

The obvious alternative keeps the fibres assignable to the query. That is the wrong way round. A shape a caller actually holds is usually *narrower* than the shape declared by its fibre, because it has literal types and a specific tag. What is wanted is the fibre this shape lies in, so the test is that the query extends the fibre's shape and not the reverse. It picks out exactly one fibre, because the shapes of a tagged union are disjoint.

This is what makes the type test in Section 3 possible. The following asserts that the positions over one prompt are exactly the fields of that prompt's report, and that a field belonging to a different fibre is rejected.

```
expect<PosAt<NodeC, { tag: "Edges" }>>({ node: 0, cw: 1, ccw: -1 });
// @ts-expect-error  hops belongs to a different fibre
expect<PosAt<NodeC, { tag: "Edges" }>>({ node: 0, cw: 1, ccw: -1, hops: 3 });
```

Every `@ts-expect-error` directive in that file asserts that the next line is an error. TypeScript reports an unused directive as an error in its own right. So the file fails if a true statement is rejected, and it also fails if a false one is accepted. Making one of the new directives true produces this.

```
TS2578 [ERROR]: Unused '@ts-expect-error' directive.
// @ts-expect-error  the cover is not the base, and it is not three arcs
```

A.2 A container that can be prompted

A runner is a container together with a program that answers it. The signature is the dependent function type: give it a shape, get back a position over *that* shape.

```
export interface Runner<C extends Cont> {
  readonly fibres?: C;
  readonly label: string;
  run<S extends Shape<C>>(shape: S, depth: number): Promise<PosAt<C, S>>;
}
```

The first field is a phantom and is never assigned. It has to exist. Everywhere else in the interface the parameter `C` occurs only inside conditional types, and TypeScript cannot infer a type argument from a conditional type. Without that line the composition of two runners silently infers the top container type, and every position downstream becomes `unknown`. The declaration exists so that `C` occurs once in a position the inference algorithm can read.

A.3 The four combinators

Each combinator is a distributive conditional type, so each computes the union of the fibres of the composite. Read the two type arguments of `Fib` as the shape line and the position line of the definition.

```
/** sum (+) — routing. Shapes are a disjoint union; each keeps its own positions. */
export type SumC<A extends Cont, B extends Cont> =
  | (A extends Fib<infer S, infer P> ? Fib<{ side: "left"; value: S }, P> : never)
  | (B extends Fib<infer S, infer P> ? Fib<{ side: "right"; value: S }, P> : never);

/** tensor (⊠) — parallelisation. Shapes multiply AND positions multiply. */
export type TensorC<A extends Cont, B extends Cont> =
  A extends Fib<infer SA, infer PA>
    ? B extends Fib<infer SB, infer PB>
      ? Fib<{ left: SA; right: SB }, { left: PA; right: PB }>
      : never
    : never;

/** product (×) — the pattern the maths predicted. Shapes multiply exactly as
 *  for the tensor; positions are a SUM. Prompt both sides, answer one. */
export type ProductC<A extends Cont, B extends Cont> =
  A extends Fib<infer SA, infer PA>
    ? B extends Fib<infer SB, infer PB>
      ? Fib<{ a: SA; b: SB }, { side: "a"; value: PA } | { side: "b"; value: PB }>
      : never
    : never;

/** sequential composition (◁) — the dependent pair, fibrewise. [comment abridged] */
export type SeqC<A extends Cont, B extends Cont> =
  A extends Fib<infer SA, infer PA>
    ? B extends Fib<infer SB, infer PB>
      ? Fib<{ first: SA; next: (r: PA) => SB }, { first: PA; second: PB }>
      : never
    : never;
```

Look at what the continuation in the last definition receives. It receives the positions of *that* shape, written `PA`, and not the union of all of the source container's positions. That is the dependency, and it is the thing the earlier note said could not be expressed.

The introduction rule is where the typing is won, because it is generic in the fibre.

```
step<SA extends Shape<A>, SB extends Shape<B>>(
  first: SA,
  next: (r: PosAt<A, SA>) => SB,
): { first: SA; next: (r: PosAt<A, SA>) => SB };
```

Since the first type parameter is inferred from the first argument, the continuation is contextually typed at that one fibre. The caller cannot see the positions of any other prompt.

A.4 Where it still leaks

The honest statement of the limit is short. Sequential composition in the source material is slightly larger than the version above. There the continuation may return a different shape of the second container for different positions of the first, so the composite position type is a genuine dependent sum over the positions,

$$\sum_{p \,:\, c.\mathrm{Pos}\ s} d.\mathrm{Pos}\,(v\,p),$$

and a dependent sum over that index is a sum over an infinite index, which does not distribute. Restricting the continuation to a single fibre is what buys the position type back, because it becomes a plain product. What is lost is a continuation that branches between shapes of the second container. Such a function has a union return type and therefore lies in no single fibre.

A.5 The handler pattern, twice

One more pattern recurs, and it is the practical payoff of the dependent lookup. A mapped type keyed by the tag forces each handler to be checked against its own concrete response type, rather than against the union of all of them.

```
type Handlers = {
  [K in MobPrompt["tag"]]: (
    p: Extract<MobPrompt, { tag: K }>,
    depth: number,
  ) => Promise<Reply<K>>;
};
```

Adding the `Deck` fibre to the container therefore produced a type error in the server until a handler for it existed, and a type error in the decoder table of the type test until a decoder for it existed. Neither had to be remembered.

B Why a separate server for each container

Every node could have been an async function. The ring would then be one process, the transition functions would be multiplications inside it, and the whole demonstration would be about a hundred lines shorter. It is not built that way, for three reasons, and the first is the one that matters.

B.1 A single process would have assumed away the question

The question under study is when local authority can be assembled into a global object, and what stops it. Electing a coordinator is one of the four repairs. If all the nodes lived in one process, the program holding them would be a coordinator by construction. It would have a complete view of every node's state, and it could compute the invariant without any node ever telling it anything.

That is why the program tags its own commands `[coordinator]` or `[in the ring]`, and the tags would be meaningless in a single process, because every command would be the first kind. The distinction is only available because the client genuinely cannot see inside a node. It has to ask, and the answer has to come back through the backward maps of morphisms it does not control.

B.2 The backward map has to be a real map

The claim of Section 1 is that the transition function lives in the backward map and in no other position. Inside one process that claim is hard to falsify and hard to demonstrate, because there is no boundary that the value must cross. A multiplication placed anywhere in the file has the same effect.

Across a process boundary the claim becomes checkable. What crosses the wire is a JSON document and a depth counter, and nothing else. The depth counter rides in an HTTP header rather than in the document, because the document is the container's shape and nothing that is not a prompt belongs in it.

```
export async function delegate(
  port: number,
  prompt: unknown,
  depth: number,
): Promise<unknown> {
  // the long comment on the in-process branch is elided here; see Appendix B
  const script = new URL("./delegate.ts", import.meta.url).pathname;
  const cmd = new Deno.Command("deno", {
    args: [
      "run", "--quiet", "--allow-net", "--allow-env",
      script, String(port), JSON.stringify(prompt), String(depth),
    ],
    stdout: "piped",
    stderr: "inherit", // the child's trace lines join ours
  });
  const { code, stdout } = await cmd.output();
  ...
}
```

Firing the low-level prompt spawns the tool that carries it to the next server. So the forward map is a process and not a function call, and control genuinely flows down. The reply is then validated at the receiving node before anything is done with it, which is the only runtime validation boundary in the program. A reply that does not validate becomes a thrown error, and the serving loop turns it into a failure that propagates back up the trace.

This is also what makes the sign visible. The trace prints one line going down and one coming up, indented by depth, and the accumulation happens between the two.

B.3 One process per hop makes the regress measurable

Async recursion in a single process does terminate the demonstration, and for the wrong reason. A recursive async function that never reaches a base case exhausts a concrete resource, and the failure is reported as a stack overflow or as memory pressure. That is a fact about the runtime. It is not a fact about the federation, and a reader would be entitled to say the program had run out of a resource rather than demonstrated an obstruction.

With one process per hop there is no shared stack. Each hop has its own, and the regress is bounded only by the machine. That is what the measurement in Section 5 is measuring, and it is why the count of live processes and the count of outbound hops in the log are the same

number. It is also why abandoning the client's request does not stop anything, which is the property a reader is least likely to guess and the one most worth showing.

An async function is a container morphism with the boundary hidden. The prompt is the argument, the response is what the promise resolves to, and the dependency of the second on the first is exactly the container's position family. What a separate process adds is not expressive power. It is the guarantee that the two maps are the only communication, that nobody is reading anybody's state, and that the composite is not secretly a single program with a global view.

There is one escape hatch, and it is off by default. A hop costs about a fifth of a second, and a parameter search that plays a hundred games of the larger project would pay that forty times per game. An environment variable keeps the HTTP hop, with the same servers, the same handlers and the same state, and skips only the spawn. No run that anybody looks at uses it, and the trace, the process counts and every number in this note were produced without it.